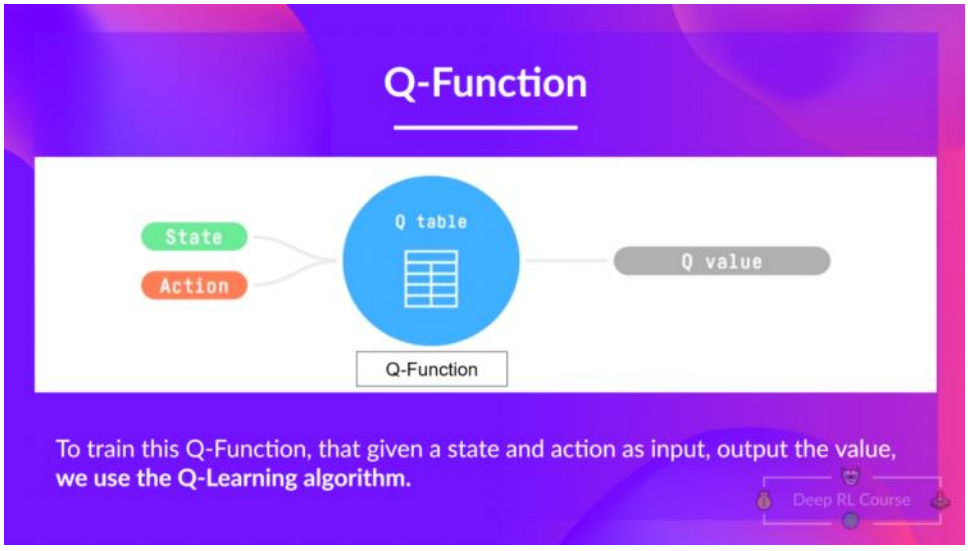
Chapter 3 Deep Q-Learning with Atari Game

12 May 2024 19:26

1. Introduction

   The state space in Atari games can contain10^9 to10^{11} states. But as we'll see, producing and updating a Q-table can become ineffective in large state space environments. So in this unit, we'll study our first Deep Reinforcement Learning agent: Deep Q-Learning. Instead of using a Q-table, Deep Q-Learning uses a Neural Network that takes a state and approximates Q-values for each action based on that state. And we'll train it to play Space Invaders and other Atari environments using RL-Zoo, a training framework for RL using Stable-Baselines that provides scripts for training, evaluating agents, tuning hyperparameters, plotting results, and recording videos.

2. From Q-learning to Deep Q-Learning
   We learned that Q-Learning is an algorithm we use to train our Q-Function, an action-value function that determines the value of being at a particular state and taking a specific action at that state.



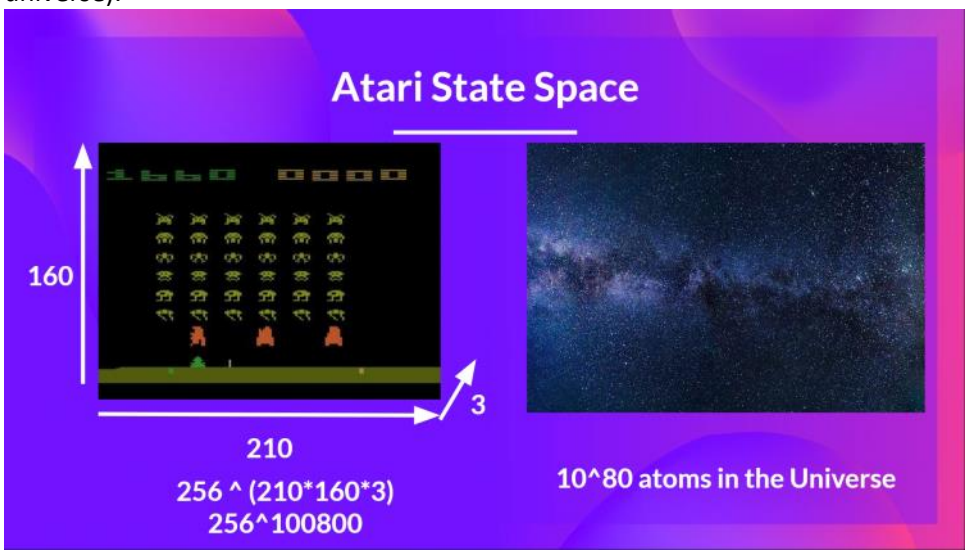
   The **Q comes from "the Quality" of that action at that state.** Internally, our Q-function is encoded by **a Q-table, a table where each cell corresponds to a state-action pair value.** Think of this Q-table as **the memory or cheat sheet of our Q-function.**
   The problem is that Q-Learning is a *tabular method*. This becomes a problem if the states and actions spaces **are not small enough to be represented efficiently by arrays and tables**. In

other words: it is **not scalable**. Q-Learning worked well with small state space environments like:

- FrozenLake, we had 16 states.
- Taxi-v3, we had 500 states.

But think of what we're going to do today: we will train an agent to learn to play Space Invaders, a more complex game, using the frames as input.

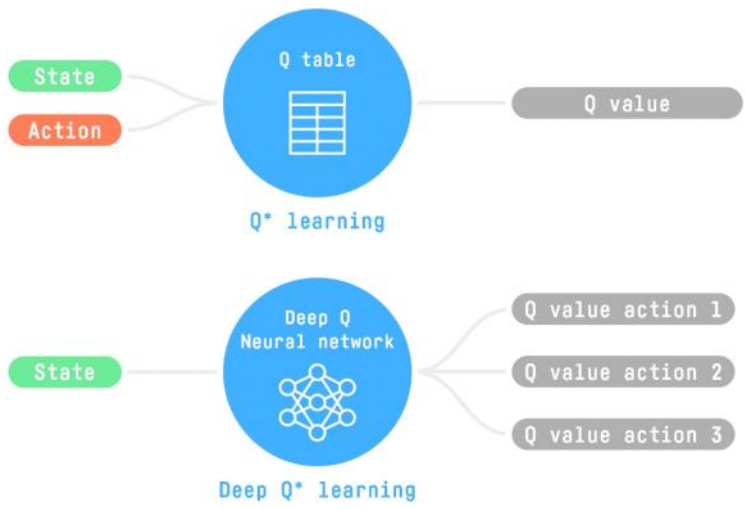
As Nikita Melkozerov mentioned, Atari environments have an observation space with a shape of (210, 160, 3), containing values ranging from 0 to 255 so that gives us $256^{210\times160\times3}=256^{100800}$ possible observations (for comparison, we have approximately10^{80} atoms in the observable universe).



A single frame in Atari is composed of an image of 210x160 pixels. Given that the images are in color (RGB), there are 3 channels. This is why the shape is (210, 160, 3). For each pixel, the value can go from 0 to 255.

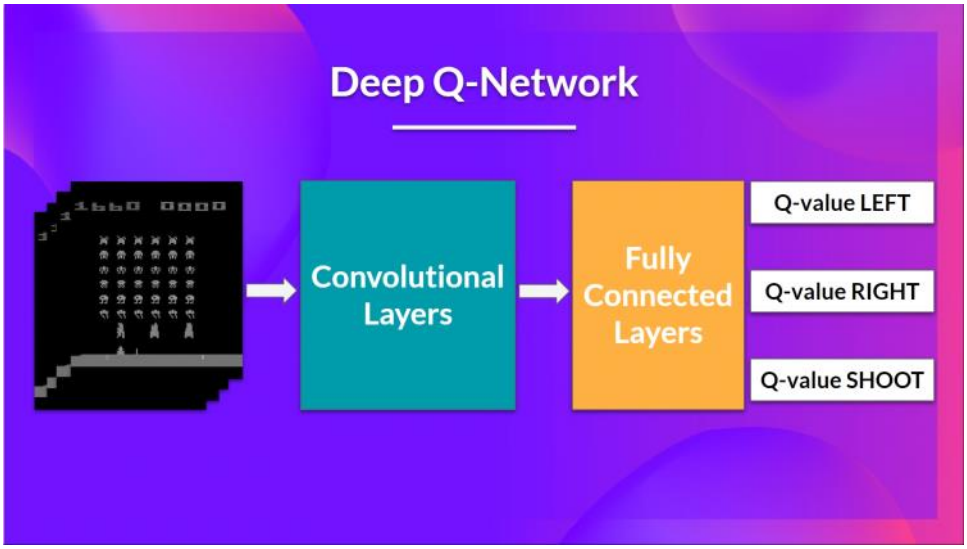
Therefore, the state space is gigantic; due to this, creating and updating a Q-table for that environment would not be efficient. In this case, the best idea is to approximate the Q-values using a parametrized Q-function $Q_\theta(s,a)$.

This neural network will approximate, given a state, the different Q-values for each possible action at that state. And that's exactly what Deep Q-Learning does.



Now that we understand Deep Q-Learning, let's dive deeper into the Deep Q-Network.

1. The Deep Q Network (DQN)
   This is the architecture of our DQN



As input, we take a **stack of 4 frames** passed through the network as a state and output a **vector of Q-values for each possible action at that state**. Then, like with Q-Learning, we just need to use our epsilon-greedy policy to select which action to take.

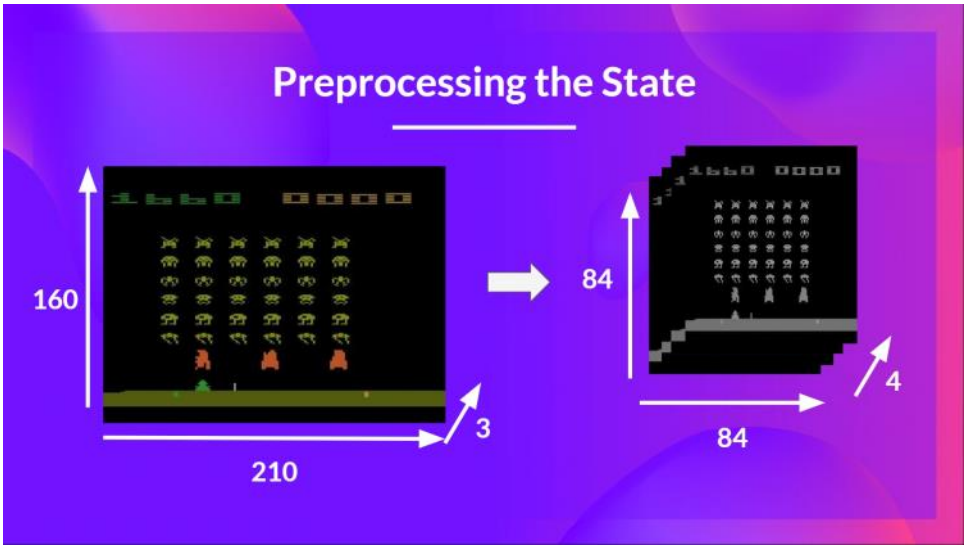
When the Neural Network is initialized, **the Q-value estimation is terrible**. But during training, our Deep Q-Network agent will associate a situation with the appropriate action and **learn to play the game well**.

2. Preprocessing the input and temporal limitation
   We need to preprocess the input. It's an essential step since we want to reduce the complexity of our state to reduce the computation time needed for training.

   To achieve this, we reduce the state space to 84x84 and grayscale it. We can do this since the colors in Atari environments don't add important information. This is a big improvement since we reduce our three color channels (RGB) to 1.

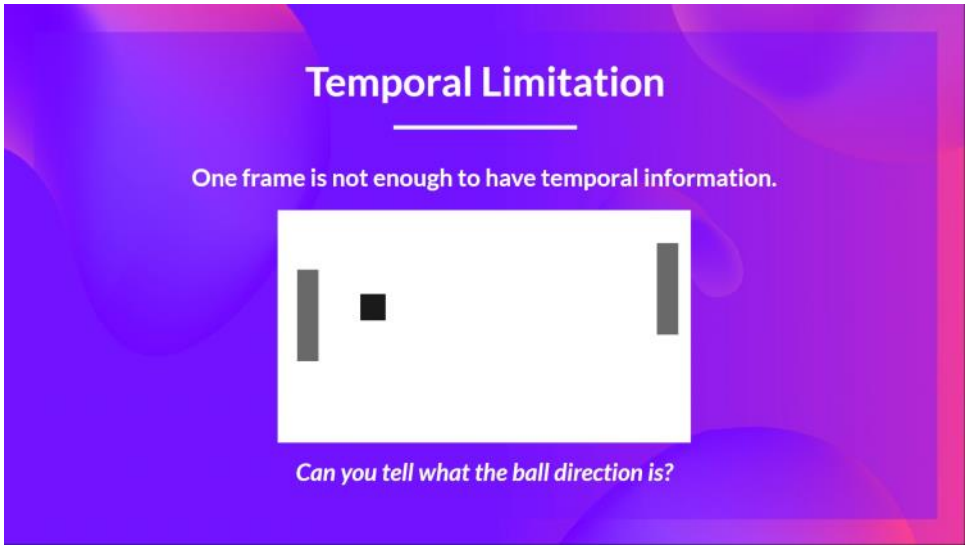
   We can also crop a part of the screen in some games if it does not contain important information. Then we stack four frames together.

We can also crop a part of the screen in some games if it does not contain important information. Then we stack four frames together.

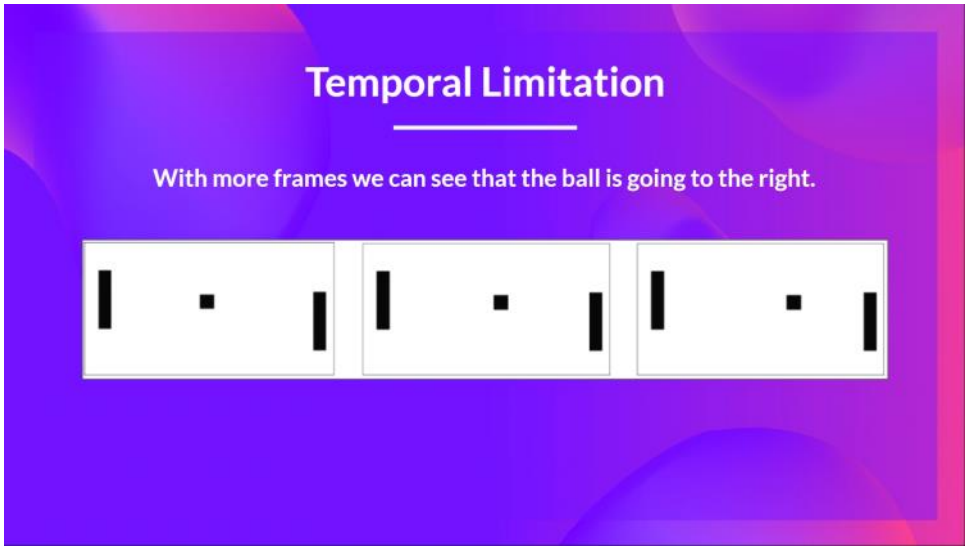


Why do we stack four frames together? We stack frames together because it helps us **handle the problem of temporal limitation**. Let's take an example with the game of Pong. When you

see this frame:



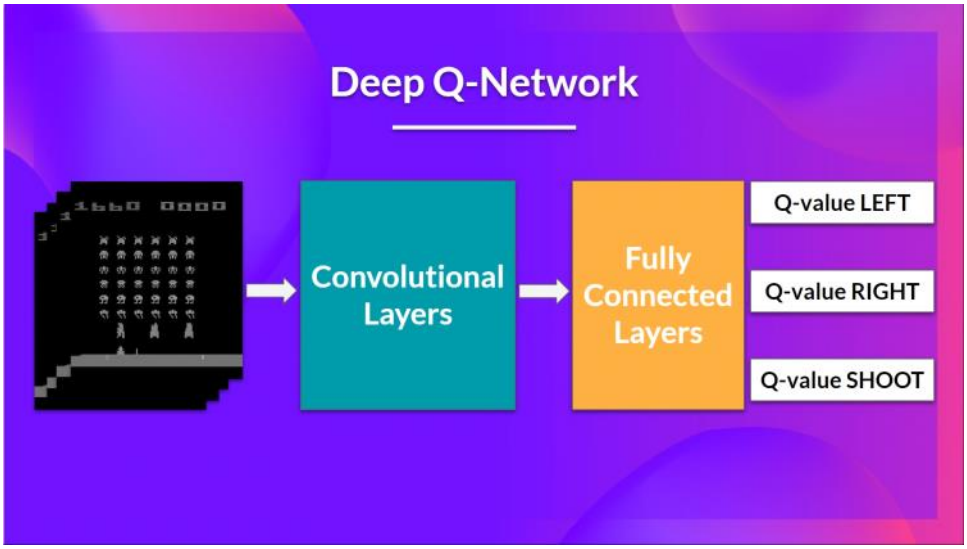
Can you tell me where the ball is going? No, because one frame is not enough to have a sense of motion! But what if I add three more frames? **Here you can see that the ball is going to the right**.



That's why, to capture temporal information, we stack four frames together.

Then the stacked frames are processed by three convolutional layers. These layers allow us to capture and exploit spatial relationships in images. But also, because the frames are stacked together, we can exploit some temporal properties across those frames.

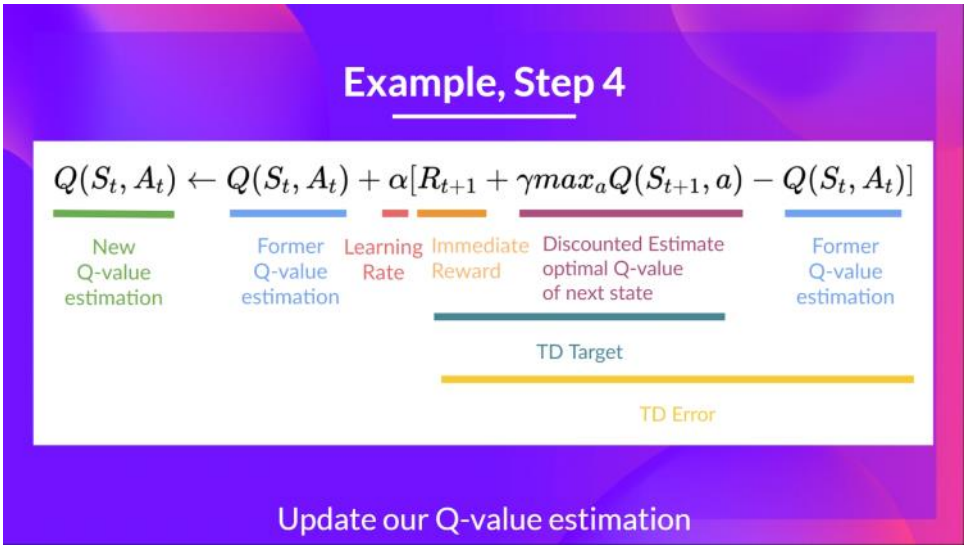
Finally, we have a couple of fully connected layers that output a Q-value for each possible action at that state.



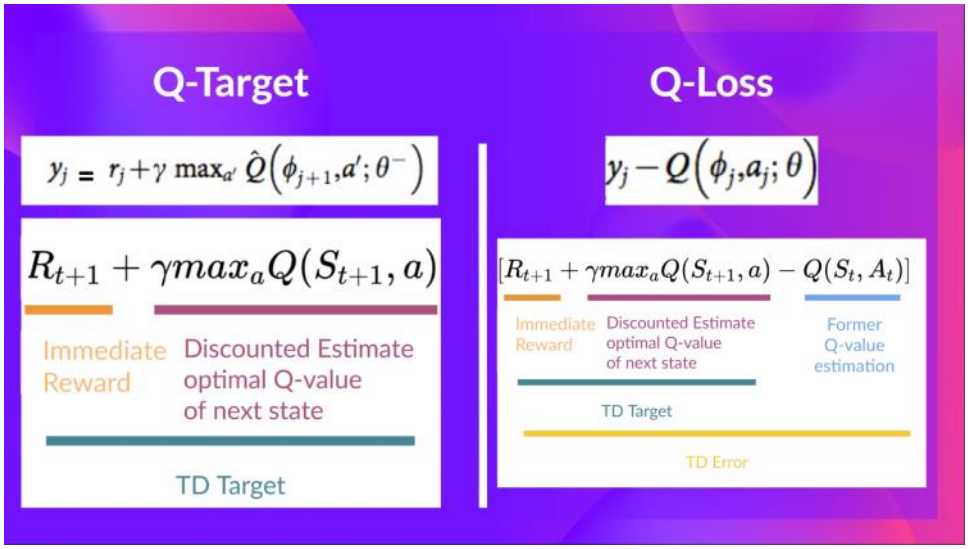
So, we see that Deep Q-Learning uses a neural network to approximate, given a state, the different Q-values for each possible action at that state. Now let's study the Deep Q-Learning algorithm.

1. The Deep Q Algorithms
   We learned that Deep Q-Learning uses a deep neural network to approximate the different Q-values for each possible action at a state (value-function estimation).

   The difference is that, during the training phase, instead of updating the Q-value of a state-action pair directly as we have done with Q-Learning:



in Deep Q-Learning, we create a loss function that compares our Q-value prediction and the Q-target and uses gradient descent to update the weights of our Deep Q-Network to approximate our Q-values better.



The Deep Q-Learning training algorithm has *two phases*:

- **Sampling**: we perform actions and **store the observed experience tuples in a replay memory**.
- **Training**: Select a **small batch of tuples randomly and learn from this batch using a gradient descent update step**.

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$
For episode $= 1, M$ **do**
  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  For $t = 1$,T **do**
    With probability ε select a random action a_t
    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
    Sampling
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $\left(y_j - Q(\phi_j, a_j; \theta)\right)^2$ with respect to the network parameters θ
    Every C steps reset $\hat{Q} = Q$
    Training
  End For
End For

This is not the only difference compared with Q-Learning. Deep Q-Learning training **might suffer from instability**, mainly because of combining a non-linear Q-value function (Neural Network) and bootstrapping (when we update targets with existing estimates and not an actual complete return).

To help us stabilize the training, we implement three different solutions:

1. *Experience Replay* to make more **efficient use of experiences**.
2. *Fixed Q-Target* **to stabilize the training**.
3. *Double Deep Q-Learning*, to **handle the problem of the overestimation of Q-values**.

Let's go through them!

1. Experience Replay to make more efficient use of experiences
   Why do we create a replay memory?
   Experience Replay in Deep Q-Learning has two functions:
   i. **Make more efficient use of the experiences during the training**. Usually, in online reinforcement learning, the agent interacts with the environment, gets experiences (state, action, reward, and next state), learns from them (updates the neural network), and discards them. This is not efficient. Experience replay helps by **using the experiences of the training more efficiently**. We use a replay buffer that saves experience samples **that we can reuse during the training.**

Experience Replay

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$
For episode $= 1, M$ **do**
  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  For $t = 1$,T **do**
    With probability ε select a random action a_t
    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^{-}) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $\left(y_j - Q(\phi_j, a_j; \theta)\right)^2$ with respect to the network parameters θ
    Every C steps reset $\hat{Q} = Q$
  End For
End For

i. **Avoid forgetting previous experiences (aka catastrophic interference, or catastrophic forgetting) and reduce the correlation between experiences**.
Catastrophic forgeing: The problem we get if we give sequential samples of experiences to our neural network is that it tends to forget **the previous experiences as it gets new experiences.** For instance, if the agent is in the first level and then in the second, which is different, it can forget how to behave and play in the first level.
The solution is to create a Replay Buffer that stores experience tuples while interacting with the environment and then sample a small batch of tuples. This prevents **the network from only learning about what it has done immediately before.**
Experience replay also has other benefits. By randomly sampling the experiences, we remove correlation in the observation sequences and avoid **action values from oscillating or diverging catastrophically.**

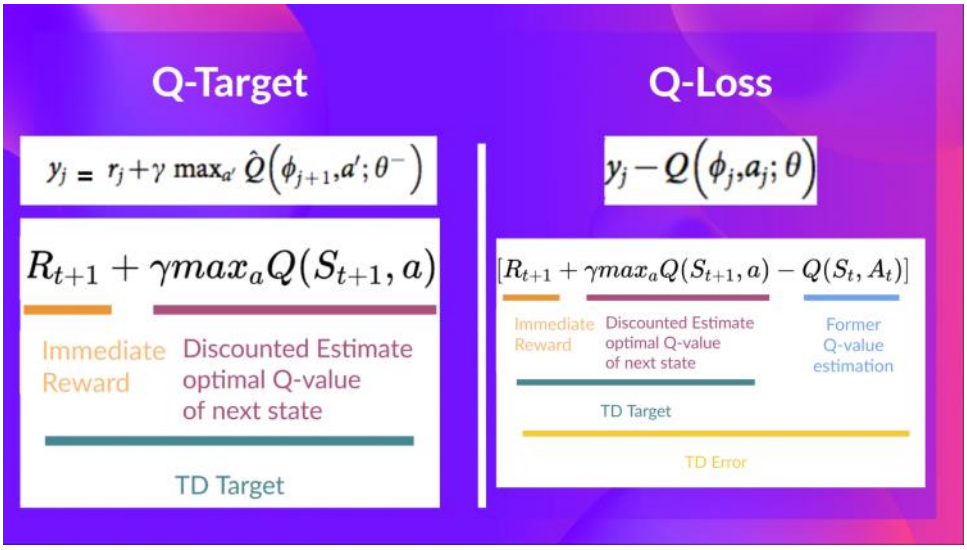
In the Deep Q-Learning pseudocode, we **initialize a replay memory buffer D with capacity N** (N is a hyperparameter that you can define). We then store experiences in the memory and sample a batch of experiences to feed the Deep Q-Network during the training phase.

Experience Replay

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$
For episode $= 1, M$ **do**
  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  For $t = 1$,T **do**
    With probability ε select a random action a_t
    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^{-}) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $\left(y_j - Q(\phi_j, a_j; \theta)\right)^2$ with respect to the network parameters θ
    Every C steps reset $\hat{Q} = Q$
  End For
End For

1. Fixed Q-Target to stabilize the training
When we want to calculate the TD error (aka the loss), we calculate the **difference between the TD target (Q-Target) and the current Q-value (estimation of Q)**.
But we **don't have any idea of the real TD target**. We need to estimate it. Using the

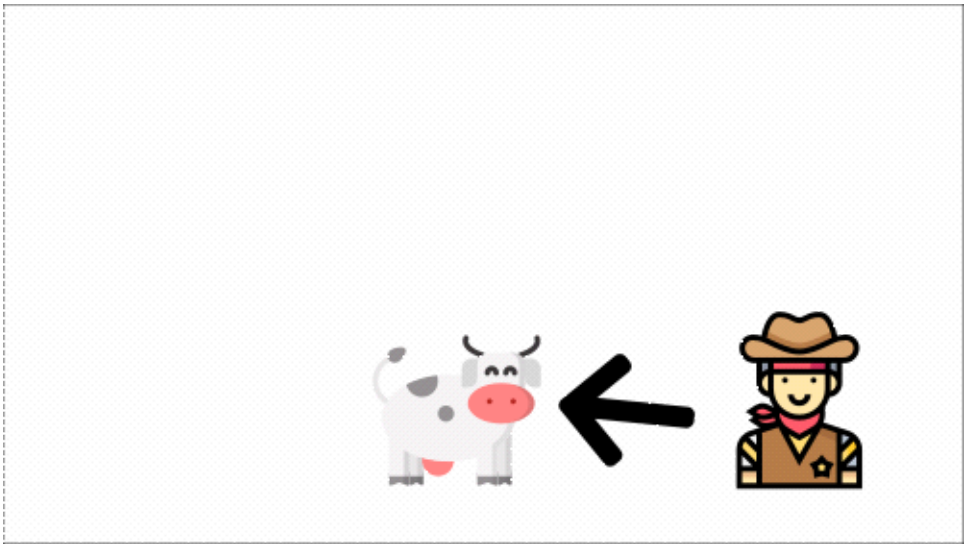
Bellman equation, we saw that the TD target is just the reward of taking that action at that state plus the discounted highest Q value for the next state.

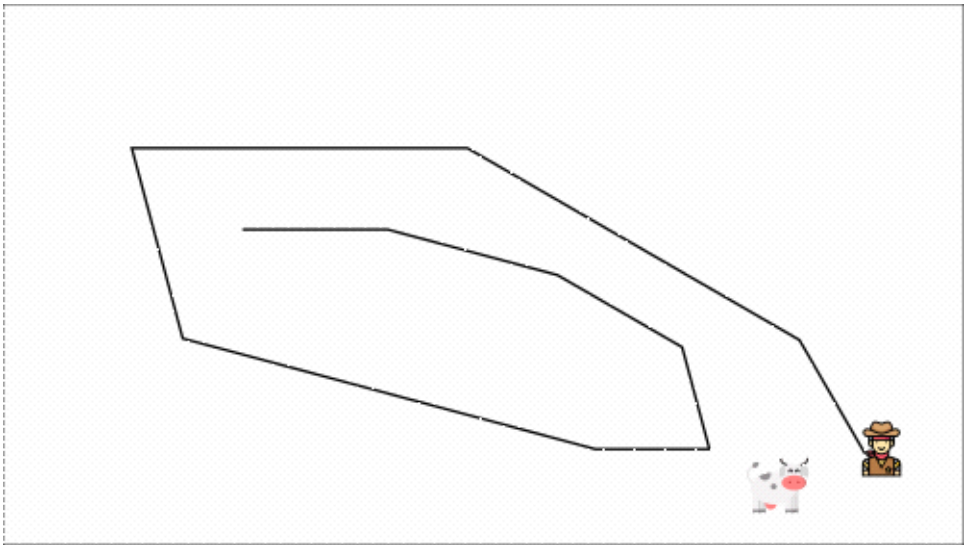


However, the problem is that we are using the same parameters (weights) for estimating the TD target and the Q-value. Consequently, there is a significant correlation between the TD target and the parameters we are changing.

Therefore, at every step of training, both our Q-values and the target values shift. We're getting closer to our target, but the target is also moving. It's like chasing a moving target! This can lead to significant oscillation in training.

It's like if you were a cowboy (the Q estimation) and you wanted to catch a cow (the Q-target). Your goal is to get closer (reduce the error)





Instead, what we see in the pseudo-code is that we:

- Use a **separate network with fixed parameters** for estimating the TD Target
- **Copy the parameters from our Deep Q-Network every C steps** to update the target network.

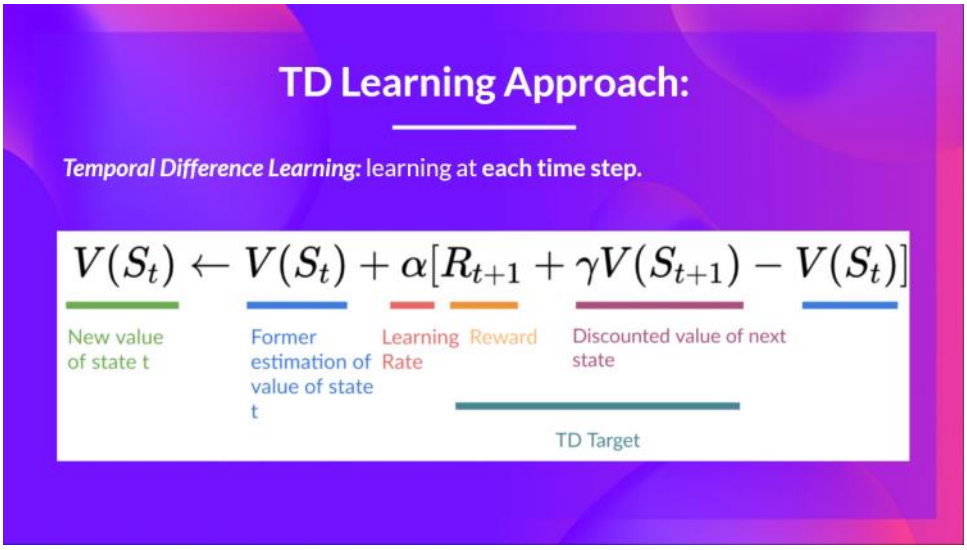
Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$
For episode $= 1, M$ **do**
  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  For $t = 1$,T **do**
    With probability ε select a random action a_t
    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^{-}) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $\left(y_j - Q(\phi_j, a_j; \theta)\right)^2$ with respect to the network parameters θ
    Every C steps reset $\hat{Q} = Q$
  End For
End For

Experience Replay

Fixed Q-Target

2. Double DQN

Double DQNs, or Double Deep Q-Learning neural networks, were introduced by Hado van Hasselt. This method **handles the problem of the overestimation of Q-values.**
To understand this problem, remember how we calculate the TD Target:



We face a simple problem by calculating the TD target: how are we sure that **the best action for the next state is the action with the highest Q-value?**
We know that the accuracy of Q-values depends on what action we tried **and** what neighboring states we explored.
Consequently, we don't have enough information about the best action to take at the beginning of the training. Therefore, taking the maximum Q-value (which is noisy) as the best action to take can lead to false positives. If non-optimal actions are regularly **given a higher Q value than the optimal best action, the learning will be complicated.**
The solution is: when we compute the Q target, we use two networks to decouple the action selection from the target Q-value generation. We:

- Use our **DQN network** to select the best action to take for the next state (the action with the highest Q-value).
- Use our **Target network** to calculate the target Q-value of taking that action at the next state.

Therefore, Double DQN helps us reduce the overestimation of Q-values and, as a consequence, helps us train faster and with more stable learning.
Since these three improvements in Deep Q-Learning, many more have been added, such as Prioritized Experience Replay and Dueling Deep Q-Learning. They're out of the scope of this course but if you're interested, check the links we put in the reading list.